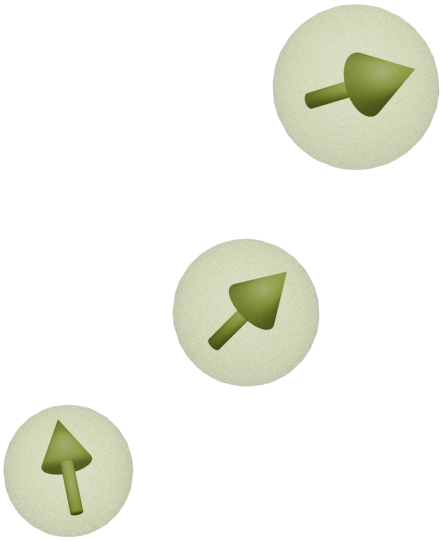
Collection of algorithms concerning spin dynamic mean-field theory

—

Documentation of the code

Timo Gräßer

2025



Contents

1 Documentation of spinDMFT

Before trying to understand the code in detail, one should understand how the method itself works. Therefore, we refer the reader to the open access article in Ref. [Grä+21] and/or dissertation in Ref. [Grä24] (Chapter 4).

The "Algorithm" directory includes five different sub-directories: "Functions", "Parameter_Space", "Run_Time_Data", "Storage_Concept" and "Time_Measure", each containing up to three files: a header file (.h) a cpp file (.cpp) and an error handling file (.h). The main code "main.cpp" is located in the directory "spinDMFT/Algorithm" and contains the essential steps done during a simulation. The functions and classes defined in the header files of the different directories are included in the main through "main_header.h". Further required header files can be found in the directory "cpp_libs", which is documented in Ch. 4.

1.1 matrices.h

1.2 Parameter_Space

- **contains:** Parameter_Space.h, Parameter_Space.cpp, PS_Error_Handling.h

This directory contains the class `ParameterSpace`, which is responsible for the handling of parameters. The class is initialized with the help of boost program options allowing to access all adjustable parameters at runtime. The parameters are set in the constructor of the class, which also contains their default values and a brief explanation within the bpo::options_description.

The parameters can be divided into three different categories: "model and physical parameters", "general numerical parameters" and "storing and naming". Model and physical parameters include for example the employed **spinmodel**, which determines the underlying spin-spin coupling of the considered system, e.g., Ising or Heisenberg. An example for a numerical parameter is the sample size `num_SamplesPerCore` per core, which is crucial for the Monte-Carlo simulation. The category storing and naming includes for example the `project_name`, which is used to subdivide data sets into different projects. To get an overview of the parameters, as well as their default values and a short explanation, one can use the help command

```
mpirun executable_XXX.out --help
```

with XXX replaced by DOUBLE or FLOAT in the terminal.

Any spinDMFT simulation requires some initial guess for the spin correlations. This guess is usually an analytic function, which can be set using the parameters **initdcorr** and **initdcorr**. Via the option **loadinit** and the parameters **initcorrsrc** and **initcorrfile**, the initial correlations can also be imported from a file. This can be helpful, for example, to efficiently redo a simulation with higher precision. Starting the simulation with some converged lower-precision correlations leads to a more rapid convergence. The importing is performed in the method `read_initial_correlations_from_file`. Note that several constraints need to be fulfilled, e.g., the imported data need to have the same symmetry type and the same floating-point data type as the current simulation. The imported correlations are stored linearly in the `ParameterSpace`. They are transformed to a `CorrelationTensor<Correlation>` (see Sec. 4.6.2) and potentially extrapolated via the function `generate_initial_spin_correlations` (see Sec. 1.5).

The class method `create_essentials_string` is called at the start of a simulation and prints some crucial parameters to the terminal.

adjust spinDMFT help command again

1.3 Run_Time_Data

- **contains:** Run_Time_Data.h, Run_Time_Data.cpp, RTD_Error_Handling.h

This directory contains the class `RunTimeData`, which is responsible for several numerical data generated in the process of the simulation. It analyzes and processes negative eigenvalues as well as the statistical error and the iteration error. The constructor initializes an instance of RunTimeData by importing some crucial numerical parameters from the ParameterSpace.

The constructor also initializes the truncation scheme `truncate_if_negative` for negative eigenvalues that potentially arise during the diagonalization of the mean-field covariance matrix. Generated eigenvalues can be handed over to the method `process_and_check_eigenvalues`, which searches for negative eigenvalues and truncates them. This method also computes the ratio of the sum of all negative eigenvalues with respect to the sum of all positive eigenvalues and terminates the simulation if the threshold value `critical_eigenvalue_ratio` is exceeded.

The constructor also initializes the single-sample square sum `sample_sqsum` $\sum_i x_i^2$ and the single-sample standard deviations `sample_stds`. The method `compute_sample_stds` obtains the Monte-Carlo sample sum $\sum_i x_i$ and employs it together with the `sample_sqsum` to compute the single-sample standard deviations $\frac{1}{M}\sum_i (x_i - \mu)^2$, where M is the sample size and μ is the average.

The method `compute_iteration_error` obtains the results of the current and previous iteration step and computes the iteration error as the maximum (over different directions) of the time average of the deviation between the inputs.

The method `terminate` returns true, if the simulation has converged, i.e., if the iteration error is below the value of `absolute_iteration_error_threshold`. Convergence is the regular scenario. The method also returns true, if the `Iteration_Limit` has been reached, or if not self-consistent iteration should be performed (bpo-option **noselfcons**).

mention the statistical error as a measure for the iteration error

1.4 Time_Measure

- **contains:** Time_Measure.h, Time_Measure.cpp

This directory contains the class `DerivedTimeMeasure`, which is responsible for measuring the duration of different steps of the simulation. Measures are performed in the main programm by calling the method `measure` providing the name of the measurement and a bool value to decide whether the result of the measurement should be directly printed to the terminal. The method `measure` is implemented in the base class `TimeMeasure`, which can be found in the directory "cpp_libs", and essentially calls the method `internal_measure`, which is implemented locally in "Time_Measure.cpp".

An important aspect complicating the time measurement is the occurrence of nested loops. To ensure that `DerivedTimeMeasure` works properly, an instance of this class needs to recognize, when it enters and when it leaves a loop (at least, if time measurements are performed within the loop). This is done using the methods `enter_loop` and `leave_loop` right before and right after the loop.

1.5 Functions

- **contains:** Functions.h, Functions.cpp, Error_Handling.h

This directory contains a collection of functions used in "main.cpp". Amongst others, it includes `generate_initial_spin_correlations`, which generates the initial spin correlations by some predefined functions (bpo-options **initdcorr** and **initdcorrsrc**) or by imported data (bpo-option **loadinit**).

The method `self_consistent_equations` performs the self consistency, i.e., it transforms the spin correlations into mean-field correlations. This involves a transformation depending on the underlying spin model, a prefactor J_{Q}^2 (quadratic coupling constant) and an optional external noise. The spin-model transformation

$$R^{\alpha\beta}(t,0) \coloneqq \langle \underline{\underline{D}}\vec{\mathbf{S}}(t) \left(\underline{\underline{D}}\vec{\mathbf{S}}(0)\right)^{\top} \rangle_{\alpha\beta} = \sum_{\rho\gamma} D^{\alpha\rho} D^{\beta\gamma} \langle \mathbf{S}^{\rho}(t)\mathbf{S}^{\gamma}(0)\rangle \tag{1.1}$$

is performed to capture the correct spin-spin coupling in the mean-fields. Here, $\underline{\underline{D}}$ is the matrix coupling one spin to another, i.e., the considered coupling is of the form $\vec{\mathbf{S}}_i^{\top} \underline{\underline{D}} \vec{\mathbf{S}}_j^{\top}$. E.g., in case of a Heisenberg coupling, $\underline{\underline{D}} = \underline{\underline{1}}$, i.e., each spin correlation $\langle \mathbf{S}^{\alpha}(t)\mathbf{S}^{\beta}(0)\rangle$

is equivalent to $R^{\alpha\beta}(t,0)$ and transforms directly into the corresponding mean-field correlation $\overline{V^{\alpha}(t)V^{\beta}(0)}^{\mathrm{mf}}$. But, for example, in case of an Ising coupling, $\langle \mathbf{S}^z(t)\mathbf{S}^z(0)\rangle$ transforms to $\overline{V^z(t)V^z(0)}^{\mathrm{mf}}$, while the other mean-field correlations strictly vanish. The spin model is implemented as a struct in "cpp_libs" under "Physics/Physics.h" and essentially contains the corresponding transformation matrix.

Besides the self consistency, the "Functions" directory also contains the essential steps of the time evolution such as the computation of propagators, the computation of $\vec{\mathbf{S}}(t)$ as well as the computation of the spin correlations. The propagators are divided into short-step propagators according to

$$\mathbf{U}(n\delta t,0)=\prod_{k=1}^{n}\mathbf{U}(k\delta t,(k-1)\delta t). \tag{1.2}$$

Since δt is small, the short-step propagators can be approximated using CFETS (commutator-free exponential time propagators). This is performed by the functions `CFET4opt_for_single_spin_one_half` and `CFET`, which are briefly explained in Sec. 4.8.2. The results of the Monte-Carlo simulation are shared among the cores via `MPI_share_results`. More details on the parallelization can be found in Sec. 1.7. The method `normalize` normalizes the spin correlations with respect to the number of samples.

1.6 Storage_Concept

- **contains:** Storage_Concept.h, Storage_Concept.cpp, STOC_Error_Handling.h

This directory contains the class `StorageConcept`, which is responsible for storing the simulation results and any attached meta data in hdf5-format. The constructor creates any required directories via `create_folder_branch` and opens a new hdf5-file via `create_file`. The parameter `m_storing_permission` ensures that only a single core (my_rank=0) operates on the file.

The method `store_main` stores the parameters, runtime data and results each in a separate group. Many of the employed storing routines are implemented in "cpp_libs" in the header "HDF5/HDF5_Routines.h". The method `store_time` stores the time measurement performed by an instance of the class `DerivedTimeMeasure`.

1.7 main.cpp and main_header.h

The main code "main.cpp" includes the previously mentioned functions and classes through the header file "main_header.h", which imports several header files and defines shorthands for certain namespaces. The main code is carried out on several cores simultaneously using MPI (message passing interface). It begins with initializing MPI and defining the integers `my_rank`, which numbers the cores, and `world_size`, which corresponds to the total number of cores. Subsequently, the time measurement, the

parameter space, the initial spin correlations, the run time data and the duration estimator are initialized. The latter estimates the duration of the Monte-Carlo simulation based on the duration of processing the first sample. The class `DurationEstimator` is implemented in a header file in "cpp_libs" and will be discussed in Sec. 4.9. The tensor of spin correlations is stored using the class `CorrelationTensor<Correlation>`, which is explained in **??**.

The self-consistent iteration starts right after the initialization. Each iteration step starts with initializing the new spin correlations to be calculated (`my_new_spin_correlations`) and generating the mean-field distribution. The latter is done by calculating the second mean-field moments self-consistently from the previously computed spin autocorrelations (`my_spin_correlations`) employing the function `self_consistent_equations`. The latter returns the mean-field covariance matrix, which is then diagonalized to obtain the eigenvalues and the orthogonal transformation matrix. The eigenvalues are truncated and checked by the runtimedata class method `process_and_check_eigenvalues` and then used to create the mean-field normal distributions in the diagonal (non-correlated) basis. Subsequently, an instance of the class `std::mt19937` (Mersenne Twister random number generator) is initialized by providing an iteration and rank-dependent seed.

This is where the Monte-Carlo simulation starts. The operations in this part of the code are dependent on the core, since the random number generator is initialized with a unique rank-dependent seed. The Monte-Carlo samples are drawn in sets of the size `num_SamplesPerSet` (bpo-option **numSamplesPerSet**). This is more efficient because performing the transformation to the non-diagonal basis for several samples at once is more cache coherent. However, for memory reasons, the size of a set of samples can be limited. The noise samples are created as an instance of the class `GaussianNoiseVectorsBlocks`, which is implemented in "cpp_libs" and will be discussed in Sec. 4.7.2. The sampling in the diagonal basis as well as the transformation to the non-diagonal basis both happens in the initialization of the class instance. The class `NoiseTensorReaderScheme` allows for efficiently accessing the created noise.

Next, the results are computed for each drawn sample. The time propagators are computed using CFETs as explained in Sec. 4.8.2. Subsequently, the time-evolved Pauli matrices are computed, which are then used to calculate the desired spin correlations. The single-sample results are simply added up in `my_new_spin_correlations`, which is later normalized. Note that also the squared sum of the single-sample results is computed. This is required for the computation of the standard deviation.

After the Monte-Carlo simulation is finished, the results are shared among the cores via `MPI_share_results`. The finalization of the iteration step includes computing the Monte-Carlo standard deviation, normalizing the spin correlations with respect to the sample size, computing the iteration error and printing some details. The iteration is terminated if the method `terminate` of the instance `my_rtdata` returns true as explained in Sec. 1.3.

Once the iteration is finished, the data are stored using the class `HDF5_Storage`. At last, MPI is finalized.

2 Documentation of CspinDMFT

Before trying to understand the code in detail, one should understand how the method itself works. Therefore, we refer the reader to the open access article in Ref. [Grä+23] and/or dissertation in Ref. [Grä24] (Chapter 5).

The "Algorithm" directory is structured very similar as the one for spinDMFT. In addition to the five sub-directories "Functions", "Parameter_Space", "Run_Time_Data", "Storage_Concept" and "Time_Measure", it also contains the directory "Mean_Field_Models". As before, the main code "main.cpp" and its header "main_header.h" as well as "matrices.h" are located in the main directory "Algorithm". Further required header files can be found in the directory "cpp_libs", which is documented in Ch. 4.

As the method CspinDMFT is an extension of spinDMFT to multiple local sites, this also applies to the code. Several objects that represented a quantity related to spin or mean-field are now obtaining one or two additional tensor dimensions representing the site indices. For example, the spin correlations stored as a `CorrelationTensor<Correlation>` in spinDMFT, are now stored as a `CorrelationCluster` of a `CorrelationTensor`, short `ClusterCorrelationTensor` or `CluCorrTen` in CspinDMFT. More information on this can be found in Sec. 4.6.3.

2.1 matrices.h

This header file defines several global blaze matrix types as well as some global matrix instances. The latter are available in most of the code and written in the beginning of the "main.cpp" using functions implemented in Sec. 2.6.

2.2 Parameter_Space

- **contains:** Parameter_Space.h, Parameter_Space.cpp, Error_Handling.h

This directory contains the class `ParameterSpace`, which is responsible for the handling of parameters. The class works in the same way as the one for spinDMFT, which is explained in Sec. 1.2.

A crucial difference to spinDMFT is the requirement for a configuration file, which is selected by the parameter **config**. The configuration file contains several important parameters such as the cluster size `num_Spins` or the local spin-spin couplings `J`. The importing of the file happens within an instance of the abstract class `MeanFieldModel`,

which is generated in the constructor of `ParameterSpace`. More information on this will be provided in Sec. 2.5. It needs to be emphasized that the specification of the configuration file is mandatory. If not provided, the simulation cannot be started.

The class methods `read_initial_correlations_from_file` and `create_essentials_string` work very similar as for spinDMFT, see Sec. 1.2.

adjust CspinDMFT help command again

2.3 Run_Time_Data

- **contains:** Run_Time_Data.h, Run_Time_Data.cpp, Error_Handling.h

This directory contains the class `RunTimeData`, which is responsible for several numerical data generated in the process of the simulation. It works very similar to the one for spinDMFT, which is explained in Sec. 1.3. An obvious difference is the extension to additional tensor dimensions for the sites. For example, the standard deviations due to the Monte-Carlo simulation are now stored as a `CluCorrTen` instead of a `CorrTen`.

An additional feature of `RunTimeData` in CspinDMFT is the possibility to determine the sample size adaptively, if this is provided via the bpo-option **adaptive** at the start of the simulation. This procedure is based on an error tolerance provided by the bpo-parameter **staterrtolerance**, which is compared to the maximum standard deviation $\sigma_{\max} = \max_{ij} \max_{\alpha\beta} \max_t \sigma_{ij}^{\alpha\beta}(t,0)$. If the tolerance is violated, the sample size is updated to an appropriate larger value in the next iteration step. Determining the sample size adaptively is especially useful in CspinDMFT because the single-sample standard deviations cannot be accessed reliably beforehand. Indeed, as explained in Ref. [Grä24] (Appendix C.2), their long-time limit can be determined beforehand from the cluster size. However, in contrast to spinDMFT, this long-time limit is often considerably exceeded at short times so that it is not really useful in practice.

Aside from the statistics, also the computation of the iteration error via the method `compute_iteration_error` is adapted. The definitions of the absolute iteration error and the relative iteration error (relative with regard to the single-sample standard deviation) read

$$\Delta I_{\mathrm{rel}} \coloneqq \sqrt{M} \max_{ij} \max_{\alpha\beta} \frac{\overline{\Delta I_{ij}^{\alpha\beta}}^{\mathrm{time}}}{\overline{\sigma_{ij}^{\alpha\beta}}^{\mathrm{time}}}, \tag{2.1a}$$

$$\Delta I_{\mathrm{abs}} \coloneqq \max_{ij} \max_{\alpha\beta} \overline{\Delta I_{ij}^{\alpha\beta}}^{\mathrm{time}}. \tag{2.1b}$$

The error to be regarded is selected via the bpo-parameter **itermode** and the compared threshold is set via **absiterror** and **reliterror**.

The method `terminate` considers an additional condition, which is checked via the method `signal_file_exists`. The latter checks, whether a specific signal file with name

"terminate_<Run_ID>" exists in the directory of the executable. The `run_ID` of a simulation is either set manually (bpo-parameter **runID**) or randomly generated. In the latter case one may read it from the terminal output at the start of the simulation. If the signal file exists, the simulation is terminated after the current iteration step. However, the results of the current iteration step are regularly stored and may be fed into a new simulation. This can be useful if a job takes an unexpectedly long time and exceeds the time limit of the employed computing resource.

2.4 Time_Measure

- **contains:** Time_Measure.h, Time_Measure.cpp

This directory contains the class `DerivedTimeMeasure`, which is responsible for measuring the duration of different steps of the simulation. It is essentially equivalent to the one for spinDMFT, which is explained in Sec. 1.4.

2.5 Mean_Field_Models

- **contains:** Mean_Field_Models.h, Mean_Field_Models.cpp, Error_Handling.h

This directory contains the header and implementation of the abstract class `MeanFieldModel`, the derived abstract class `MultiModel` and the further derived class `CorrelationReplicaModel`. These classes are mainly responsible to handle the couplings between the spins and mean-fields as well as the self-consistency conditions. The reason for having this specific class structure and not a single class is to allow for considering different kinds of mean-field models in future implementations. The class `CorrelationReplicaModel` considers individual potentially correlated mean-fields for each spin. Other models may for example consider some or all spins to couple to the same mean-field.

The constructor of `MeanFieldModel` is called within the constructor of `ParameterSpace` and obtains several parameters from the latter including the mean-field model to consider (only one available currently) and the name of the configuration file. The latter is read out in `read_configuration_from_file` obtaining important parameters such as the cluster size `num_Spins`, the spin lengths `spin_float_list`, the spin-spin couplings `J_linearized`, the spin-mean-field couplings `Jsq_linearized` and, if provided, also the `correlation_categories` to be considered for the self consistency. The specification of `correlation_categories` is necessary if one intends to use only a subset of the correlations, that are available on the cluster, for the self consistency. In this case, only this subset is computed and superposed within the self-consistency conditions in each iteration step. The remaining correlations are completely ignored. At the end of the constructor, several parameters that have been read from the configuration file are handed back to the `ParameterSpace` in order to be processed and/or stored there as well.

The virtual method `compute_Hamiltonian` of `MultiModel` obtains the value of all mean-fields at a specific time and computes the full Hamiltonian (spin-spin couplings and spin-mean-field couplings) at this time point.

The virtual method `interpret_model_specific_parameters` in `CorrelationReplicaModel` delinearizes the imported spin-mean-field couplings stored in `J_linearized` so that the latter can be properly interpreted in the self-consistency conditions. The delinearized couplings are stored as a `SymmMatrixOfMatrix` (four-dimensional tensor). Each submatrix with pair index i, j contains the information of how the spin correlations are superposed in order to construct the mean-field moment $\overline{V_i(t)V_j(0)}^{\mathrm{mf}}$. A matrix element k, l of the submatrix corresponds to the weight $\left(J^{\mathrm{mf}}_{ij,kl}\right)^2$ of the spin correlation $\langle \mathbf{S}_k(t)\mathbf{S}_l(0)\rangle$.

The virtual method `rescale` rescales the spin-mean-field couplings and spin-spin couplings according to the parameters `rescale_meanfield` and `rescale_spinspin`.

The virtual method `self_consistency` transforms spin correlations of the cluster environment into the covariance matrix of the mean-fields. To this end, the spin correlations are first transformed according to the underlying spin-model (Heisenberg, XXZ, …) via

$$R^{\alpha\beta}_{kl}(t,0) \coloneqq \langle \underline{\underline{D}}\vec{\mathbf{S}}_k(t)\left(\underline{\underline{D}}\vec{\mathbf{S}}_l(0)\right)^{\top}\rangle_{\alpha\beta} = \sum_{\rho\gamma} D^{\alpha\rho}D^{\beta\gamma}\langle \mathbf{S}^{\rho}_k(t)\mathbf{S}^{\gamma}_l(0)\rangle, \tag{2.2}$$

where $\underline{\underline{D}}$ is the matrix capturing the spin-spin coupling $\vec{\mathbf{S}}_i^{\top}\underline{\underline{D}}\vec{\mathbf{S}}_j$. This step is essentially the same as for spinDMFT, see Sec. 1.5. Subsequently, a `CluCorrTen` containing the second mean-field moments is generated. Each mean-field moment with index pair i, j results from a superposition of the rotated spin correlations with index pairs k, l according to

$$\overline{V^{\alpha}_i(t)V^{\beta}_j(0)}^{\mathrm{mf}} = \sum_{kl}\left(J^{\mathrm{mf}}_{ij,kl}\right)^2 R^{\alpha\beta}_{kl}(t,0). \tag{2.3}$$

The weights $\left(J^{\mathrm{mf}}_{ij,kl}\right)^2$ are read from `J_mf_sq`. The mean-field moments are then written into a covariance matrix using a `CorrelationVectorTensorClusterFillingScheme` as explained in Sec. 4.7.3. Note that there is just a single covariance matrix for all mean-fields together, since they are correlated and cannot be treated independently.

2.6 Functions

- **contains:** Functions.h, Functions.cpp, Error_Handling.h

This header file contains a collection of functions used in "main.cpp". The function `write_general_spin_matrices` writes the global matrices representing $\mathbf{1}$, $\mathbf{0}$ and $\mathbf{S}^{\alpha}_i$ for $i \in 1, \ldots, N_{\Gamma}$ and $\alpha \in \{x, y, z\}$. All matrices are defined on the full cluster Hilbert space.

Th spin matrices are created by a tensor product of the local matrices at each site as explained in Sec. 4.5.1.

The function `write_cluster_Hamiltonian` writes the cluster Hamiltonian, i.e., everything but the spin-mean-field couplings, to the global matrix `H_CLUSTER`. This includes local spin terms like chemical shifts or quarupolar couplings as well as the couplings between the spins of the cluster. This requires not only the global spin matrices $\mathbf{S}_i^\alpha$ and $\mathbf{S}_j^\beta$, but also the coupling factor J_{ij} stored in `J` and the spin-model factor $D^{\alpha\beta}$ stored in `spinspin_cmodel` of the parameter space.

The function `generate_initial_environment_spin_correlations` generates the initial guess for the spin correlations inserted to the self consistency similar to the analogue function in spinDMFT, see Sec. 1.5. The correlations are either set by predefined analytic functions, contained in `init_diag_corr` and `init_nondiag_corr` of the parameter space, or by imported correlations depending on the value of `init_corr_mode`.

The function `propagate` updates the time evolution operator by multiplying a short-step propagator to it according to

$$\mathbf{U}(t+\delta t,0)=\mathbf{U}(t+\delta t,t)\mathbf{U}(t,0). \tag{2.4}$$

The latter is approximated by CFET-2 [AF11] according to

$$\mathbf{U}(t+\delta t,t)\approx \mathrm{e}^{-\mathrm{i}\frac{\delta t}{2}(\mathbf{H}(t)+\mathbf{H}(t+\delta t))}. \tag{2.5}$$

The evaluation of the matrix exponential is a bit involved. It can be done either by taylor expansion, which is not recommended because of large numerical errors, or by diagonalization of the exponent.

The function `compute_spin_correlations` computes the spin correlations for all required pairs of sites k,l and pairs of directions α,β.

After the Monte-Carlo simulation is finished, the results are shared among the cores via `MPI_share_results`.

The function `normalize_and_fill_time_zero` sets the value of the spin correlations at time zero according to

$$\langle \mathbf{S}_i^\alpha(0)\mathbf{S}_j^\beta(0)\rangle=\delta_{kl}\delta^{\alpha\beta}\frac{S(S+1)}{3} \tag{2.6}$$

and normalizes the value at other times according to the sample size.

2.7 Storage_Concept

- **contains:** Storage_Concept.h, Storage_Concept.cpp, Error_Handling.h

The storage concept is very similar to the one for spinDMFT, see Sec. 1.6. One difference is the necessity to store a `CluCorrTen`, which is done by the function `store_correlation_tensor_cluster`. This method stores a `CorrTen` for each pair of sites i,j naming it according to "i-j".

2.8 main.cpp and main_header.h

The main code is structured similar to the one for spinDMFT, see Sec. 1.7. Hence, only the differences are highlighted in this section. The initialization works essentially the same as for spinDMFT. An additional step is the construction of several global matrices using the functions `write_general_spin_matrices` and `write_cluster_Hamiltonian`, which are explained in Sec. 2.6.

The beginning of each self-consistency iteration step is also very similar to the one for spinDMFT. It involves creating the mean-field covariance matrix using the self-consistency equations and subsequently diagonalizing it. The new spin autocorrelations to be computed in the iteration step are initialized to zero and stored in `my_new_spin_correlations`. In opposition to spinDMFT, they now represent a `CluCorrTen` instead of a `CorrTen` due to the additional site indices. It should be noted that these correlations are initialized with the parameter `correlation_categories` from the parameter space. Correspondingly, if specified in the configuration file, only a certain subset of correlations is considered and thus computed in the Monte-Carlo simulation as explained in Sec. 2.5.

The Monte-Carlo simulation is also very similar to the one for spinDMFT. The scheme to read noise vectors from `GaussianNoiseVectorsBlocks` is now a `NoiseTensorClusterReaderScheme` instead of a `NoiseTensorReaderScheme` due to the additional site index of the noise. The time evolution is structured a bit differently compared to the one for spinDMFT. Instead of computing all propagators at once, only a single propagator is stored and updated in a loop over the time steps. The method `compute_Hamiltonian` computes the Hamiltonian for a given set of mean-fields at a specific time (obtained from the reader scheme) as explained in Sec. 2.5. This `new_Hamiltonian` and the `old_Hamiltonian`,i.e., the one from the previous time step, are inserted into the function `propagate`, which updates the time evolution operator by multiplying a short-step propagator to it as described in Sec. 2.6. Subsequently, the function `compute_spin_correlations` computes the spin correlations for all required pairs of sites and pairs of directions and the specific time point implied by the current propagator.

The finalization of each iteration step is also very similar to the one for spinDMFT. One difference is that the estimated standard deviations now need to be processed in case of an adaptive sample size. This is done by the method `compute_and_process_sample_stds`, which is explained in Sec. 2.3.

Once the iteration is finished, the data are stored using the class `HDF5_Storage`. At last, MPI is finalized.

config file creation!!!!!!!!!!!!!!! chemical shifts for CspinDMFT not yet implemented

3 Documentation of nl-spinDMFT

This code is used to perform simulations of a cluster in a mean-field background. A crucial aspect is that the spin correlations defining the mean-fields are generated beforehand, i.e., this code does not contain any self-consistency anymore. Therefore it can be used to perform the second step of non-local spinDMFT (nl-spinDMFT). We refer the reader to the open access article in Ref. [GHU24] and/or dissertation in Ref. [Grä24] (Chapter 6) for details about the method.

Note that the code is also helpful if one intends to compute completely local quantities like autocorrelations as long as the mean-fields are *a priori* given. This is the case, for example, if the mean-field background results from a different spin sort for which the self-consistency can be solved independently beforehand using spinDMFT or CspinDMFT.

mention p-spinDMFT?

3.1 matrices.h

This header file defines several global blaze matrix types as well as some global matrix instances. The latter are available in most of the code and written in the beginning of the "main.cpp" using functions implemented in Sec. 3.6.

3.2 Parameter_Space

- **contains:** Parameter_Space.h, Parameter_Space.cpp, Error_Handling.h

This directory contains the class `ParameterSpace`, which is responsible for the handling of parameters. The class works in the same way as the one for CspinDMFT, which is explained in Sec. 2.2.

big difference: read_initial_correlations_from_file

3.3 Run_Time_Data

- **contains:** Run_Time_Data.h, Run_Time_Data.cpp, Error_Handling.h

This directory contains the class `RunTimeData`, which is responsible for several numerical data generated in the process of the simulation. It is essentially a reduced version of the analogous class in CspinDMFT, which is explained in Sec. 2.3. Since nl-spinDMFT

contains no self consistency, any methods or parameters related to iteration or termination are removed.

3.4 Time_Measure

- **contains:** Time_Measure.h, Time_Measure.cpp

This directory contains the class `DerivedTimeMeasure`, which is responsible for measuring the duration of different steps of the simulation. It is almost equivalent to the one for spinDMFT, which is explained in Sec. 1.4.

3.5 Mean_Field_Models

- **contains:** Mean_Field_Models.h, Mean_Field_Models.cpp, Error_Handling.h

This directory contains the header and implementation of the abstract class `MeanFieldModel`, the derived abstract class `MultiModel` and the further derived class `CorrelationReplicaModel`. It is structured very similar to the one for CspinDMFT, which is explained in Sec. 2.5. Only the differences are highlighted in this section.

The method `read_configuration_from_file` imports the additional parameter `import_only`, which correlations should be imported as environment correlations. The parameter contains the corresponding site double indices.

The virtual method `interpret_model_specific_parameters` in `CorrelationReplicaModel` delinearizes the imported spin-mean-field couplings stored in `J_linearized` so that the latter can be properly interpreted in the self-consistency conditions. Note that although the nl-spinDMFT contains no self consistency loop, one still needs conditions to determine the second mean-field moments from the imported environment correlations. As these conditions are related to the common self-consistency conditions, they are named self-consistency conditions in the code as well. In CspinDMFT, the spin-mean-field couplings are stored as a matrix of a matrix, since each mean field moment with index pair i, j was constructed from a superposition of the N_Γ^2 spin correlations on the cluster requiring N_Γ^2 weights $\left(J^{\mathrm{mf}}_{ij,kl}\right)^2$. In nl-spinDMFT, however, the environment correlations can be completely unrelated to those of the cluster so that the cluster site indices k, l have no meaning for them. Of course, each environment correlation can still be associated with a pair of site indices in the full spin system, but these numbers are not relevant for the nl-spinDMFT code. Therefore, the spin-mean-field couplings are stored as a matrix of a vector instead. Each second mean-field moment with index pair i, j is constructed from a superposition of the `num_Import` imported environment correlations correlations with weights $\left(J^{\mathrm{mf}}_{ij,p}\right)^2$, where p numbers the environment correlation.

The virtual method `self_consistency` of `CorrelationReplicaModel` `CorrTenList` instead of a `CluCorrTen`

$$selfconswithpinsteadofkl \tag{3.1}$$

3.6 Functions

- **contains:** Functions.h, Functions.cpp, Error_Handling.h

3.7 Storage_Concept

- **contains:** Storage_Concept.h, Storage_Concept.cpp, Error_Handling.h

3.8 main.cpp and main_header.h

4 Documentation of cpp_libs

The directory “cpp_libs” contains a collection of header files that are used in the different codes. The headers already contain some explanatory comments. This chapter shall provide a more detailed description.

Several header files require the global type definitions `RealType` and `Dim`. As will be explained below, `RealType` is set by macro variables in the header file “Globals/Types.h”, which is included in the affected header files. `Dim` has to be set manually as `constexpr size_t` before certain header files are included. Otherwise there will be a compilation error. The affected header files are exclusively in the `Physics` folder and the requirement for the `Dim` variable is always clarified in the initial comment line.

Some directories contain a header file named “Error_Handling.h” or similar, in which the error messages of some runtime errors are implemented.

symmetry_type

4.1 Globals

4.1.1 Types.h

This short header file defines `uint`, the floating point data type `RealType` and the corresponding complex data type `ComplexType`. `RealType` is either set to its default value `double`, or, if the corresponding macro `USE_FLOAT` is defined, to `float`.

4.1.2 MPI_Types.h

This short header file defines the MPI data type that corresponds to `RealType` and is used for any MPI communications of this specific data type.

4.1.3 Matrix_Types.h

This short header file defines aliases of several basic blaze vector and matrix types according to Tab. 4.1.

Alias	Definition
`Rotation`	`blaze::StaticMatrix<RealType,3UL,3UL>`
`FieldMatrix`	`Rotation`
`DiagonalRotation`	`blaze::DiagonalMatrix<blaze::StaticMatrix<RealType,3UL,3UL>`
`RealSpaceVector`	`blaze::StaticVector<RealType,3UL,blaze::columnVector>`
`FieldVector`	`RealSpaceVector`
`GeneralMatrix`	`blaze::DynamicMatrix<ComplexType,blaze::rowMajor>`
`HermitianMatrix`	`blaze::HermitianMatrix<GeneralMatrix>`
`DiagonalMatrix`	`blaze::DiagonalMatrix<GeneralMatrix>`
`SparseGeneralMatrix`	`blaze::CompressedMatrix<ComplexType,blaze::rowMajor>`
`SparseHermitianMatrix`	`blaze::HermitianMatrix<SparseGeneralMatrix>`

Table 4.1: List of blaze aliases.

4.2 File_Management/File_Management.h

This header file contains several functions for file management. The function `Orientation()` provides the relative path from the current location to the location of the file ".orientation", which is located in the main directory of the repository. The functions `create_folder` and `create_folder_tree` create a folder or a whole list of folders from given paths, respectively. The function `get_system_name()` returns the name of the operating system as `std::string`. This name is typically stored in the hdf5-files created by the different codes.

4.3 HDF5/HDF5_Routines.h

This header file contains a collection of functions for storing and reading data in hdf5 format. Simple storing routines include `store_scalar`, `store_list`, `store_string` and `store_string_list`, which perform what their name suggests. They all obtain the ID of the hdf5-file or group, where the attribute data should be stored, as well as a name and the data itself. The functions `store_scalar` and `store_list` are templated, so that their hdf5 data type has to be deduced. This is done by the `get_H5_Type` method, which translates several standard C++ data types into hdf5 data types.

The header file also contains the tensor storing routines `store_ND_tensor`, where N can be replaced by 2, 3 or 4 (a general implementation has not yet been attempted). The function obtains a file/group ID, a name, the base type of the tensor as well as the tensor itself. Before storing, the tensor is linearized via the function `linearize_ND_tensor_to_vector`, where N is again 2, 3 or 4. Note that these functions are all templated, i.e., the provided tensor can be quite general. However, the tensor and all of its subtensors need to contain the `size()` method as well as standard begin and end iterators `begin()`, `end()`, `cbegin()` and `cend()`.

The functions `import_scalar`, `import_string` and `import_list` are used for importing attribute data from an hdf-file providing the file/group ID, the data name and the variable, to which the data should be written, by reference.

The function `import_ND_tensor_linearized` allows for importing a linearized tensor providing the ID, the name and the linearized data holder by reference. Optionally, one can also hand over a list, to which the tensor dimensions should be written, by reference.

4.4 Standard_Algorithms

4.4.1 Standard_Algorithms.h

This header file contains some standard algorithms for container iteration and sorting. The functions `for_2each`, `for_3each` and `for_n_each` are essentially extensions of `std::for_each` to simultaneous iterations over several containers at once. While `for_n_each` is the most general, it may not be the easiest to read in the code because the lambda function and the iterators are swapped in the function parameters.

The template function `argsort` returns a list of indices corresponding to the sorted order of an inserted `std::vector` and a comparison function which is `std::less` by default. The vector is not changed in the process. The function `argsort_flip` works the same way but returns the flipped index list. The functions `argsort_extended` and `argsort_flip_extended` apply `argsort` or `argsort_flip`, respectively, and subsequently group all equal elements according to an equality condition that is also handed over and return the result as a vector of index lists. The function `argmin` simply returns the first index to the minimum element of a container.

The function `possibilities` recursively finds all (N over k) possibilities to draw k out of N elements.

4.4.2 Numerics.h

This header file contains some standard numerical algorithms. The function `extrapolate` linearly extrapolates a vector from an old equidistant discretization to a new equidistant discretization. The functions `mean` and `stddev` compute mean and standard deviation of the inserted container.

Aside from that, the header file contains several functions concerning floating point arithmetic such as the functions `is_equal` and `is_equal_soft` for comparing floating point numbers or `is_zero` and `is_zero_soft` to check whether a floating point number is zero (very close to it). Based on these, there are also the functions `cast_if_real` and `cast_if_real_soft` for casting complex to real values if the imaginary part is negligible.

4.4.3 Print_Routines.h

This header file contains some standard routines for printing to the terminal and creating or manipulating instances of `std::string`.

4.5 Matrices

4.5.1 Matrices.h

This header file contains some routines for basic matrix operations. The routine `count_zeros` counts the number of zero elements according to `is_zero` in a matrix, which can be important to decide whether the use of a compressed (sparse) matrix is reasonable. The function `cast_to_sparse` casts a blaze dense matrix to a blaze compressed matrix manually by removing any values that are numerically zero according to `is_zero`. The function `truncate_sparse` removes any numerically zero elements of a blaze compressed matrix to save additional memory.

Important for CspinDMFT and nl-spinDMFT is the function `Nth_order_tensor_product`, which obtains an `std::vector` of matrices and returns the total tensor product

$$\underline{\underline{M}} = \bigotimes_{i=1}^{N} \underline{\underline{M}}_i = \underline{\underline{M}}_1 \otimes \underline{\underline{M}}_2 \otimes \cdots \otimes \underline{\underline{M}}_N. \tag{4.1}$$

The matrices must be square, but their dimensions do not need to be equal. The function makes use of the sub-routine `tensor_product`, which computes the tensor product of two general square matrices.

4.5.2 Diagonalization.h

This header file contains the diagonalization functions `diagonalize_real` and `diagonalize_cplx` obtaining the blaze matrix to be diagonalized as well as the eigenvalues and orthogonal/unitary transformation by reference. If blaze diagonalization (via LAPACK) is available, the functions simply apply the standard blaze diagonalization function `blaze::eigen`. If the macro `EIGEN` is defined instead, the header includes the eigen libraries and automatically uses the `Eigen::SelfAdjointEigenSolver` for diagonalization. The code blocks for the eigen diagonalization are a bit larger as they are adapted to the syntax of eigen and also require casting from blaze to eigen types and vice versa.

4.6 Observables

4.6.1 Correlations.h

This header contains the class `CorrelationVector`, which is essentially a wrapper around `std::vector`. Despite standard element access and iterators, this class also allows some standard vector operations such as multiplication by scalar or adding or subtracting another vector. Morevover, a correlation can be constructed providing a function and an equidistant discretization (fixed δt).

4.6.2 Tensors.h

This header contains the template class `CorrelationTensor`, which serves for straightforwardly storing and accessing quantities in three-dimensional real space. The class is used in all three codes. It is a wrapper around `std::vector` and contains `Correlation` elements. Each element is assigned to two direction indices $\alpha, \beta \in \{0(x), 1(y), 2(z)\}$, which are stored in the index-pair list `m_direction_pairs`. A key ingredient of the class is the `symmetry_type` containing the information about which elements of the tensor are equivalent or zero. It is set in the constructors and can be accessed by the method `get_symmetry`. The currently availabe symmetry types are listed in Tab. 4.2.

symmetry type	**size**	**correlations**
A	1	$G_1 = G^{xx} = G^{yy} = G^{zz}$, rest is zero
B	2	$G_1 = G^{xx} = G^{yy}$, $G_2 = G^{zz}$, rest is zero
C	4	$G_1 = G^{xx} = G^{yy}$, $G_2 = G^{xy}$, $G_3 = G^{yx}$, $G_4 = G^{zz}$, rest is zero
D	9	all nine correlations are stored

Table 4.2: Symmetry types and their corresponding sizes as well as the contained correlations.

Except for the default constructor, the constructors all require the `symmetry_type` as an argument (in some constructors indirectly). The last constructor is rather special and allows to create the `CorrelationTensor` from another `CorrelationTensor` and a so-called transformation scheme, which determines each `Correlation` element of the new `CorrelationTensor` from a certain transformation of the inserted `CorrelationTensor`. This functionality is used in the implementation of the self-consistency conditions.

The contained `Correlation` elements can be accessed by the get functions, e.g., `get_xx`, by iterators or by the operator `[]`, which obtains the linearized index. Trying to explicitly access a correlation that is zero within the current symmetry type leads to either a run-time error, or, if the method `zero_according_to` is available in the template class `Correlation`, returns a corresponding implemented zero value.

The iterator functions `iterate` and `const_iterate` allow for iterating over all non-zero tensor elements appying a lambda function that is handed over. The lambda function

takes not only the `Correlation` element but also the corresponding direction index pair as arguments. With the extended iterator functions `iterate2` and `const_iterate2`, one can also iterate over two `CorrelationTensors` of the same symmetry type simultaneously. In this case, the lambda function obtains three arguments, namely the two `Correlation` elements and the corresponding direction index pair.

4.6.3 Clusters.h

This header file contains the class `CorrelationCluster`, which is a wrapper around `std::vector` of correlation objects, for example, `CorrelationTensor`. The class is essential in the codes CspinDMFT and nl-spinDMFT

to be finished

4.7 Multivariate_Gaussian

4.7.1 Multivariate_Gaussian.h

This header contains the classes `CovarianceMatrix`, `DiagonalBasisNormalDistributions` and `GaussianNoiseVectors`.

The class `CovarianceMatrix` is a wrapper around `SymmetricMatrix` therefore enforcing the contained matrix to be symmetric. However, positivity, which is another neccessary condition for a covariance matrix, is not enforced and has to be checked manually. The matrix can be filled within construction or after default construction in several different ways. Note that a non-diagonal matrix element M_{ij} only needs to be filled in once at (i, j) and is implicitly filled into (j, i) to due to the usage of `blaze::SymmetricMatrix`. The covariance matrix may be filled by providing a matrix via `fill_correlationmatrix` or a vector via `fill_correlationvector`. The latter is typically used in spinDMFT and its extensions, where the covariance matrix is built from spin correlations, which are stored as vectors in time. A vector $(x_1, x_2, x_3, \dots)^\top$ are inserted according to the following scheme:

$$\underline{\underline{M}} = \begin{pmatrix} x_1 & x_2 & x_3 & \dots \\ x_2 & x_1 & x_2 & \dots \\ x_3 & x_2 & x_1 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \tag{4.2}$$

Matrices and vectors may also be inserted into a subblock of the matrix providing the start row and column via the methods `fill_correlationvector_to_subtriangle`, `fill_correlationmatrix_to_subsquare` or `fill_symmetriccorrelationmatrix_to_diagonal_subblock`. The method `fill_correlationvector_to_subtriangle` fills the triangle of a

subblock as well as the mirrored triangle in the covariance matrix. The method `fill_correlationmatrix_to_subsquare` fills a subblock of the covariance matrix as well as the mirrored subblock in the covariance matrix. If the subblock is around the diagonal, one may use the more efficient method `fill_symmetriccorrelationmatrix_to_diagonal_subblock` instead.

The method `diagonalize` computes the eigenvalues and eigenvectors, which are provided by reference as function arguments.

The class `DiagonalBasisNormalDistributions` is a wrapper around an `std::vector` of `std::normal_distribution`. The latter are initialized with zero mean and variances provided in the constructor or in the `fill` method in case of default construction. The normal distributions can be accessed by standard vector iterators.

The class `GaussianNoiseVectors` is a wrapper around an `blaze::DynamicMatrix` and essentially stores a set of Gaussian-distributed noise vectors in matrix format. The noise is constructed providing an instance of `DiagonalBasisNormalDistributions`, a pseudo-random generator `std::mt19937` and, if unequal to 1, the number of noise samples in the set. The noise can be transformed to another basis via `basis_transform` providing an orthogonal transformation matrix (orthogonality is not checked). The noise can be accessed by `()` operators or standard matrix iterators.

4.7.2 Multivariate_Gaussian_Blocks.h

This header extends "Multivariate_Gaussian.h" by providing the classes `CovarianceMatrixBlocks`, `DiagonalBasisNormalDistributionsBlocks` and `GaussianNoiseVectorsBlocks`, which are each wrappers around `std::vector` of the corresponding classes in "Multivariate_Gaussian.h" and contain essentially the same methods. A special new method in `CovarianceMatrixBlocks` is `fill_from_scheme`, which allows a template class `FillingScheme` to fill the covariance matrix. This is used in the implementation of the filling schemes in "Multivariate_Gaussian/Symmetry_Schemes.h".

4.7.3 Symmetry_Schemes.h

The other header files in this subdirectory do not directly contain or require any information about the `symmetry_type`. The latter enters through filling and reader schemes implemented in this header file. It contains the template classes `CorrelationVectorTensorFillingScheme`, `CorrelationVectorTensorClusterFillingScheme`, `CorrelationMatrixTensorFillingScheme`, `NoiseTensorReaderScheme` and `NoiseTensorClusterReaderScheme`, which essentially do what their names suggest.

The filling schemes are used to fill the covariance matrix with correlations according to a specific `symmetry_type` and, in case of `CorrelationVectorTensorClusterFillingScheme`, also according to the number of spins in the cluster (this is required for CspinDMFT and nl-spinDMFT). Generally, the covariance matrix is filled according to

$$\underline{\underline{M}} = \begin{pmatrix} v^{xx} & v^{xy} & v^{xz} \\ v^{yx} & v^{yy} & v^{yz} \\ v^{zx} & v^{zy} & v^{zz} \end{pmatrix} \tag{4.3a}$$

with

$$v^{\alpha\beta} = \begin{pmatrix} v^{\alpha\beta}(0,0) & v^{\alpha\beta}(0,\delta t) & v^{\alpha\beta}(0,2\delta t) & \dots \\ v^{\alpha\beta}(\delta t,0) & v^{\alpha\beta}(\delta t,\delta t) & v^{\alpha\beta}(\delta t,2\delta t) & \dots \\ v^{\alpha\beta}(2\delta t,0) & v^{\alpha\beta}(2\delta t,\delta t) & v^{\alpha\beta}(2\delta t,2\delta t) & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \tag{4.3b}$$

$$= \begin{pmatrix} v^{\alpha\beta}(0,0) & v^{\alpha\beta}(0,\delta t) & v^{\alpha\beta}(0,2\delta t) & \dots \\ v^{\alpha\beta}(\delta t,0) & v^{\alpha\beta}(0,0) & v^{\alpha\beta}(0,\delta t) & \dots \\ v^{\alpha\beta}(2\delta t,0) & v^{\alpha\beta}(\delta t,0) & v^{\alpha\beta}(0,0) & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix} \tag{4.3c}$$

and

$$v^{\alpha\beta}(t_1,t_2) \coloneqq \overline{V^{\alpha}(t_1)V^{\beta}(t_2)}^{\mathrm{mf}}, \tag{4.3d}$$

where time-translation invariance is assumed in the last step. In case of several mean-fields (CspinDMFT and nl-spinDMFT), the submatrices are built according to

$$V^{\alpha\beta} = \begin{pmatrix} v_{11}^{\alpha\beta}(0,0) & v_{11}^{\alpha\beta}(0,\delta t) & \dots & v_{12}^{\alpha\beta}(0,0) & v_{12}^{\alpha\beta}(0,\delta t) & \dots \\ v_{11}^{\alpha\beta}(\delta t,0) & v_{11}^{\alpha\beta}(0,0) & \dots & v_{12}^{\alpha\beta}(\delta t,0) & v_{12}^{\alpha\beta}(0,0) & \dots \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots \\ v_{21}^{\alpha\beta}(0,0) & v_{21}^{\alpha\beta}(0,\delta t) & \dots & v_{22}^{\alpha\beta}(0,0) & v_{22}^{\alpha\beta}(0,\delta t) & \dots \\ v_{21}^{\alpha\beta}(\delta t,0) & v_{21}^{\alpha\beta}(0,0) & \dots & v_{22}^{\alpha\beta}(\delta t,0) & v_{22}^{\alpha\beta}(0,0) & \dots \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots \end{pmatrix} \tag{4.4a}$$

with

$$v_{ij}^{\alpha\beta}(t_1,t_2) \coloneqq \overline{V_i^{\alpha}(t_1)V_j^{\beta}(t_2)}^{\mathrm{mf}}. \tag{4.4b}$$

Symmetries often allow to store the `CovarianceMatrixBlocks` more efficiently. Depending on the `symmetry_type`, the `CovarianceMatrixBlocks` are stored according to Tab. 4.3

The class `CorrelationVectorTensorFillingScheme` is essentially build to fill an instance of `CorrelationTensor<CorrelationVector>` into the `CovarianceMatrixBlocks` following the above scheme. The `CorrelationVectorTensorClusterFillingScheme` simply extends this to filling an instance of `CorrelationCluster<CorrelationTensor<CorrelationVector>`. The `CorrelationMatrixTensorFillingScheme` provides an alternative way of filling the covariance matrix if time-translation invariance is not valid. This

symmetry Type	block sizes	subblocks
A	1	$v_1 = v^{xx} = v^{yy} = v^{zz}$, rest is zero
B	2	$v_1 = v^{xx} = v^{yy}$, $v_2 = v^{zz}$, rest is zero
C	2	$v_1 = \begin{pmatrix} v^{xx} & v^{xy} \\ v^{yx} & v^{yy} \end{pmatrix}$, $v_2 = v^{zz}$, rest is zero
D	1	the whole matrix is stored in a single block

Table 4.3: Block sizes of the covariance matrix and their corresponding subblocks for the different symmetry types.

is not further discussed because it is not available in the current implementation of the code.

Generating `GaussianNoiseVectorsBlocks` is not very complex and therefore not performed by a filling scheme but simply by using the class constructors in combination with the function `return_num_noises_per_block` implemented in this header file. Depending on the `symmetry_type`, the `GaussianNoiseVectorsBlocks` are stored according to Tab. 4.4.

symmetry Type	block sizes	subblocks
A	1	$\mathcal{V}_1 = \begin{pmatrix} \mathcal{V}^x & \mathcal{V}^y & \mathcal{V}^z \end{pmatrix}$
B	2	$\mathcal{V}_1 = \begin{pmatrix} \mathcal{V}^x & \mathcal{V}^y \end{pmatrix}$, $\mathcal{V}_2 = \mathcal{V}^z$
C	2	$\mathcal{V}_1 = \begin{pmatrix} \mathcal{V}^x \\ \mathcal{V}^y \end{pmatrix}$, $\mathcal{V}_2 = \mathcal{V}^z$
D	1	$\mathcal{V}_1 = \begin{pmatrix} \mathcal{V}^x \\ \mathcal{V}^y \\ \mathcal{V}^z \end{pmatrix}$

Table 4.4: Block sizes and subblocks of the Gaussian noise vectors for different symmetry types.

where

$$\mathcal{V}^\alpha = \begin{pmatrix} V^\alpha_{(1)}(0) & \dots & V^\alpha_{(M)}(0) \\ V^\alpha_{(1)}(\delta t) & \dots & V^\alpha_{(M)}(\delta t) \\ \vdots & \dots & \vdots \end{pmatrix} \tag{4.5}$$

is the noise set with M samples for direction α and all time steps. In case of several mean-fields (CspinDMFT and nl-spinDMFT), the noise is stored according to

$$\mathcal{V}^{\alpha} = \begin{pmatrix} V^{\alpha}_{1,(1)}(0) & \dots & V^{\alpha}_{1,(M)}(0) \\ V^{\alpha}_{1,(1)}(\delta t) & \dots & V^{\alpha}_{1,(M)}(\delta t) \\ \vdots & \dots & \vdots \\ V^{\alpha}_{2,(1)}(0) & \dots & V^{\alpha}_{2,(M)}(0) \\ \vdots & \dots & \vdots \end{pmatrix}. \tag{4.6}$$

Note that the covariance matrix, or to be precise, its eigenvalues and orthogonal transformation are required to generate `GaussianNoiseVectorsBlocks` explaining why the latter must have the same block structure.

Last but not least, this header contains the noise reader schemes `NoiseTensorReaderScheme` and `NoiseTensorClusterReaderScheme`. These each carry a `std::shared_ptr` to a `GaussianNoiseVectorsBlocks` and allow to straightforwardly and efficiently read elements from it using pointers. The read-out is done time step by time step using the method `read` which, in case of `NoiseTensorReaderScheme` (for spinDMFT), returns the mean-field at the next time step as `Vec` (3D `blaze::StaticVector`). In case of `NoiseTensorClusterReaderScheme` (for CspinDMFT or nl-spinDMFT), the method returns all mean-fields at the next time step as `VecVec` (`std::vector` of `Vec`).

4.8 Physics

4.8.1 Physics.h

This header contains several practical classes used in the different codes. Amongst others, it contains the classes `DiagonalSpinCorrelation` and `NonDiagonalSpinCorrelation`, which generate `std::function` objects that are used to define the initial spin correlations used in the algorithms. Providing an step width δt, the method `create_discretization` evaluates the function at a discrete equidistant set of time steps.

The class `SpinModel` is used to define the model by which the spins couple to one another. The coupling is always bilinear and of the form $\vec{\mathbf{S}}_i^{\top} \underline{\underline{D}} \vec{\mathbf{S}}_j$ with some transformation matrix $\underline{\underline{D}}$, which is not necessarily a rotation matrix. The transformation matrix is saved as a class member `coupling_matrix`, which can be directly accessed. It required for setting up self-consistency equations in all three algorithms and to set up the local cluster Hamiltonian in case of CspinDMFT and nl-spinDMFT.

The class `MagneticField` is used to define a static global magnetic field and the class `ChemicalShift` adds a static magnetic field in z direction, which may vary locally. The class `Noise` stores the 3x3 variance tensor of a Gaussian static noise source. The class `ExtraInteraction` provides a local interaction term in dependence of the spin matrices $\mathbf{S}^x$, $\mathbf{S}^y$ and $\mathbf{S}^z$. An example for this is a quadrupolar interaction term.

4.8.2 CFET.h

• **requires:** `HermitianMatrix ZERO` and `HermitianMatrix IDENTITY`

This header contains implementations of Commutator-free exponential time (CFET) propagators. For more information on the math behind this, see **??**. Mostly, CFET-2, which is equivalent to a first-order truncation of the Magnus expansion, is used because the resulting numerical error is already of the same order as the one from the numerical evaluation of the occurring time integrals. Specifically for a single spin with $S = \frac{1}{2}$, an optimized CFET-4 with three exponentials is used because the evaluation of matrix exponentials is particularly easy for this case due to the formula

$$e^{-i\vec{F}\cdot\vec{\sigma}} = \cos(F)\sigma^0 - i\frac{\sin(F)}{F}\vec{F}\cdot\vec{\sigma}. \tag{4.7}$$

This is implemented in the function `CFET4opt_for_single_spin_one_half`. For higher spin values and/or several spins as in CspinDMFT and nl-spinDMFT, the evaluation of matrix exponentials is more involved. Currently, the evaluation is done by diagonalization of the exponent which is straightforward although certainly not the most efficient way.

4.8.3 Spin_Geometries.h

• **requires:** `constexpr size_t Dim`

This header file contains the classes `InhomogeneousEnsemble` and `Lattice`, which can be used to generate a spin `Ensemble`, i.e., a list of spin positions.

The class `InhomogeneousEnsemble` generates random spin positions in a `Dim`-dimensional cube. The cube center is at the origin $(0, 0, \dots)^\top$. The class includes two different constructors. The first creates the ensemble from a given seed, a given number of spins N and, optionally, a density $\rho = N/L^{\mathrm{Dim}}$ (L is the cube side length), which is one by default. The second constructor creates the ensemble from a given seed, a given number of spins N, a given set of fixed spin positions that are automatically added, a minimum distance between any pair of spins and, optionally, another `Constraint` (`std::function` returning a `bool` given a `Spin`) and a density, which is one by default. The generated `Ensemble` can be printed and returned from the class.

The class `Lattice` generates a spin ensemble on a lattice in `Dim` dimensions. The constructor requires a `Unit_Cell`, the number of cells in each direction and the lattice vectors to shift the unit cell. A `Unit_Cell` is a `std::vector` of `Group`, which is itself a `std::vector` of `Spin`. This structure can is advantageous if the later-required clusters are chosen manually. A `Group` (or later `Cluster`) does not necessarily form a cell, by which a total lattice can be generated. A unit cell of combined groups, however, is able to do so. The cell and group structure is removed once the ensemble is returned. However, the central group appears first in the generated `Ensemble` and can thus be

easily found. Aside from the `Lattice` class, this header also contains the functions `create_SimpleCubic` and `create_Triangular` each creating a specific lattice.

4.8.4 Coulings.h

- **requires:** `constexpr size_t Dim`

This header file contains several functions to compute spin-spin couplings in specific systems. Each of these functions obtains the position of the two spins and potentially additional parameters. Providing a coupling function and a set of spin positions, the general function `compute_coupling_matrix` returns all spin-spin couplings as a `blaze::SymmetricMatrix`.

4.8.5 Clusterization.h

- **requires:** `constexpr size_t Dim`

This header file contains several different schemes to divide a spin ensemble into cluster and environment, which is required for setting up CspinDMFT. The schemes obtain amongst others an `Ensemble` which is a list of spin positions and return two `Cluster`, which are also positions lists, as well as an `IndexList` containing the indices of the spins in the cluster within the ensemble. The function `auto_clusterize` simply takes the first N_Γ list entries of the ensemble and puts them into the cluster. Via `manually_add_spin_to_cluster` one can add an additional spin by index to an already clusterized ensemble. The remaining functions implement schemes of the categories central-spin based, cluster based and hybrid, which are explained in Ref. [Grä24] (Appendix D). Some of the schemes are defined differently for lattices and inhomogeneous systems. Moreover the cluster size N_Γ may be fixed manually or be chosen by some optimization algorithm. The preferred scheme according to Ref. [Grä24] is `clusterize_hybrid` or `clusterize_hybrid_vsize` in case of a variable cluster size. It may be used for lattices and inhomogeneous systems.

4.8.6 Mean_Field_Couplings.h

- **requires:** `constexpr size_t Dim`

This header file contains several algorithms to compute spin-mean-field couplings such as the quadratic coupling constant.

Contained are also the Correlation Replica Approximation (CRA) and Correlation Replica (CR) scheme, which compute coupling tensors in inhomogeneous systems and lattices, respectively, from the given cluster, environment and the coupling function. These schemes are required to initialize the CspinDMFT simulation. The CRA for inhomogeneous systems returns the weights of each spin correlation of the cluster in the second mean-field moments. These weights are also stored as a symmetric matrix of matrices according to

$$\begin{pmatrix} J_{11}^{\mathrm{CRA}} & J_{12}^{\mathrm{CRA}} & \dots \\ J_{21}^{\mathrm{CRA}} & J_{22}^{\mathrm{CRA}} & \dots \\ \vdots & \vdots & \ddots \end{pmatrix} \quad \text{with} \quad J_{ij}^{\mathrm{CRA}} = \begin{pmatrix} J_{ij,11}^{\mathrm{CRA}} & J_{ij,12}^{\mathrm{CRA}} & \dots \\ J_{ij,21}^{\mathrm{CRA}} & J_{ij,22}^{\mathrm{CRA}} & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}, \tag{4.8}$$

where the element $J_{ij,kl}^{\mathrm{CRA}}$ describes the weight of the spin correlation $\langle S_k(t)S_l(0)\rangle$ in the second mean-field moment $\overline{V_i(t)V_j(0)}^{\mathrm{mf}}$. The CR scheme removes symmetry-equivalent correlations to arrive at a minimal set of distinguishable spin correlations. Only these specific spin correlations contribute to the coupling tensor and are thus required to close the self consistency. The CR scheme returns a list of index pairs each referring to one of these spin correlations. It also returns a symmetric matrix of matrices containing the weights of all possible spin correlation in the corresponding second mean-field moment similar to the CRA scheme. Correlations that are not occurring in the index list simply lead to zero elements in the coupling tensor and will be ignored in the CspinDMFT simulation. An explanation of the functionality of the schemes and how the specific couplings are computed is provided in Ref. [Grä24] (Section 5.1.4).

The header also contains the function `compute_Jbath_sq_simple` used to compute the coupling tensor for nl-spinDMFT. The function returns a symmetric matrix of vectors according to

$$\begin{pmatrix} J_{11}^{\mathrm{bath}} & J_{12}^{\mathrm{bath}} & \dots \\ J_{21}^{\mathrm{bath}} & J_{22}^{\mathrm{bath}} & \dots \\ \vdots & \vdots & \ddots \end{pmatrix} \quad \text{with} \quad J_{ij}^{\mathrm{bath}} = \begin{pmatrix} J_{ij,(0)}^{\mathrm{bath}} \\ J_{ij,(1)}^{\mathrm{bath}} \\ \vdots \end{pmatrix}, \tag{4.9}$$

where the element $J_{ij,p}^{\mathrm{bath}}$ captures the weight of the spin correlation $\langle \mathbf{S}_m(t)\mathbf{S}_n(0)\rangle$ with $p = \{m, n\}$ in the second mean-field moment $\overline{V_i(t)V_j(0)}^{\mathrm{mf}}$. Note that the indices m, n are not necessarily indices of spins that are symmetry-equivalent to spins in the cluster. They refer to bath spins which may not be related to the cluster spins at all. The function `compute_Jbath_sq_simple` is only implemented for the case of a single spin autocorrelation entering the second mean-field moments, i.e., each vector contains only a single element $J_{ij,(0)}^{\mathrm{bath}}$. An explanation of the scheme and how the elements are computed is provided in Ref. [Grä24] (Section 6.1.2 and specifically Eq. (6.5)).

squares everywhere!!!

4.8.7 Spin.h

This header file contains several functions dealing with the spins and their associated Hilbert space. The function `write_spin_matrices` fills the local spin matrices S^x, S^y and S^z, which are handed over by reference, depending on the spin length. For example

in case of $S = \frac{1}{2}$, the matrices are set to the standard Pauli matrices. Similarly, the functions `create_local_zero` and `create_local_identity` return the local zero and identity matrix in dependence of the spin length.

The function `create_zero_and_identity` returns the zero and identity matrix for a given Hilbert space dimension (the Hilbert space may contain several spins). The function `create_linear_spin_term` creates a spin matrix on a Hilbert space with several spins providing the spin lengths as well as the corresponding spin index i and direction α of the spin operator. To this end, the function creates a list of the local matrices at each site. These are then combined to a single matrix by the tensor product according to

$$\mathbf{S}_i^\alpha = \underbrace{\mathbf{1} \otimes \ldots}_{1 \to i-1} \otimes \underbrace{\mathbf{S}^\alpha}_{i} \otimes \underbrace{\cdots \otimes \mathbf{1}}_{i+1 \to N} \tag{4.10}$$

spin lengths are not yet in the config file creation mechanism check all config file creations

4.8.8 Random.h

This header file is used to generate seeds for the random number generators used in the codes. The function `generate_seed` generates a seed from a string input, which either contains the word “random” or corresponds to some unsigned integer. The generated seed is shared among the cores via MPI. The function `throw_seed` obtains this seed to generate a local seed which depends on `my_rank`, i.e., the core, and `num_Iteration`, i.e., the number of the current iteration step.

4.9 Time_Measure/Time_Measure.h

This header file contains the classes `DurationQuantity`, `TimeMeasure`, `SimpleDuration Estimator` and `DurationEstimator`. `DurationQuantity` simply stores a time duration with a corresponding name.

`TimeMeasure` is an abstract base class used in all of the codes to track and store the duration of different processes. Particularly, the class contains a mechanism to measure the duration of subprocesses in nested loops. For this to work, loops have to be entered and left using the methods `enter()` and `leave()`. A duration measurement is carried out with `measure` providing the name of the duration and, optionally, whether the duration should be printed to the terminal. The method can be called within or outside of loops. One might want to use the method `update_time()`, which ensures that the subsequent duration measurement is done with respect to the time of this function call. Otherwise, duration measurements are always done with regard to the previous measurement. The time measurement is finalized via the method `stop()`. Generally, two different types of measures are distinguished: the scalar `m_global_measure_time` and

the list `m_tmp_measure_times`. The global measure time is updated with each `measure` call that happens outside of any loop, while a measurement within a loop updates the corresponding element in `m_tmp_measure_times`. A call of `measure` invokes a call of the virtual undefined method `internal_measure`. To be able to use the abstract class `TimeMeasure`, one has to define a derived class and define this virtual method. In the derived class constructor, one should define the different time measurements one intends to perform in the main code. The method `internal_measure` should manage, how the measured times are stored.

The classes `SimpleDurationEstimator` estimates the duration of a loop after the first pass. To this end, it obtains the size of the loop in the constructor and the duration of the first loop pass via the `obtain` method. If desired, this method also shows a progress bar in the terminal. The class `DurationEstimator` is an extension of this to nested loops. It obtains the size of all involved loops in the constructor and obtains the durations of a single pass for each loop.

Bibliography

The abbreviation scheme of the references is based on the authors last names and the year. In case of three or less authors, the first letters of the last names are appended and, in case of more than three authors, the first three letters of the first authors last name including a '+' symbol are taken.

[AF11] A. Alvermann and H. Fehske. "High-order commutator-free exponential time-propagation of driven quantum systems." Journal of Computational Physics **230**, 5930 (2011).

[GHU24] T. Gräßer, T. Hahn, and G. S. Uhrig. "Microscopic understanding of NMR signals by dynamic mean-field theory for spins." Solid State Nuclear Magnetic Resonance **132**, 101936 (2024).

[Grä+21] T. Gräßer et al. "Dynamic mean-field theory for dense spin systems at infinite temperature." Physical Review Research **3**, 043168 (2021).

[Grä+23] T. Gräßer et al. "Understanding the dynamics of randomly positioned dipolar spin ensembles." Physical Review Research **5**, 043191 (2023).

[Grä24] T. Gräßer. *Dynamic mean-field theory for simulating the infinite-temperature dynamics of spin ensembles.* PhD thesis (Technische Universität Dortmund, 2024).